

Documentacion tecnica completa del CRM HPD

Fecha: 18 de septiembre de 2026

1. Objetivo

Este documento explica como esta construido el CRM HPD: carpetas, archivos, responsabilidades, flujo de datos, base de datos, seguridad, integraciones, despliegue recomendado y pendientes tecnicos.

El manual para usuarios del CRM esta en MANUAL_TECNICO_CRM_HPD.md.

2. Resumen tecnico

El proyecto es una aplicacion ASP.NET Core con frontend estatico en wwwroot, backend por controladores API, Entity Framework Core para SQL Server, integraciones con Meta/WhatsApp/Instagram/Facebook, almacenamiento de archivos con Cloudinary y configuracion local editable en App_Data.

Componentes principales:

- ASP.NET Core: servidor web, APIs, autenticacion y middleware.
- Entity Framework Core: acceso a SQL Server.
- SQL Server LocalDB: base local del CRM.
- Frontend HTML/CSS/JS: interfaz del usuario.
- App_Data: configuracion local de bot e integraciones.
- ngrok: tunel de desarrollo para webhooks publicos.
- Cloudinary: almacenamiento externo de archivos.
- Meta Graph API: WhatsApp, Facebook e Instagram.

3. Estructura general de carpetas

CRM.Data/

Proyecto principal ASP.NET Core.

Contiene:

- Program.cs: arranque de la aplicacion.
- appsettings.json: configuracion base local.
- Controllers/: endpoints HTTP.
- Data/: DbContext.
- Models/: entidades de base de datos.
- Services/: logica de negocio e integraciones.
- DTOs/: objetos de entrada/salida.
- Extensions/: configuracion de servicios, middleware y seguridad.
- Migrations/: migraciones de Entity Framework.
- Startup/: inicializacion de base y datos iniciales.
- wwwroot/: interfaz web.
- App_Data/: archivos JSON locales de configuracion.

tools/

Herramientas locales para generar documentacion:

- markdown_to_html.js: convierte Markdown a HTML.
- markdown_to_pdf.js: convierte Markdown a PDF.

Documentos raiz

- AVANCE_PROYECTO_CRM.md: avance general del proyecto.
- MANUAL_TECNICO_CRM_HPD.md: manual operativo para personal.
- DOCUMENTACION_TECNICA_COMPLETA_CRM_HPD.md: este documento.

4. Archivos de arranque y configuracion

CRM.Data/Program.cs

Archivo de entrada de ASP.NET Core.

Responsabilidades:

- Crear el builder.
- Registrar servicios.
- Registrar base de datos.
- Registrar autenticacion por cookies.
- Registrar rate limiting.
- Configurar CORS.
- Configurar headers de seguridad.
- Inicializar base de datos.
- Publicar archivos estaticos.
- Activar autenticacion y autorizacion.
- Mapear controladores.
- Mapear fallback hacia index.html.

CRM.Data/appsettings.json

Configuracion local del proyecto.

Puede incluir:

- Connection string.
- Versiones de API.
- Tokens de integracion.
- App IDs.
- Flags de envio real.
- Configuracion Cloudinary.

Recomendacion de seguridad:

- No usar secretos reales en repositorios.
- En produccion usar variables de entorno o secret manager.
- Regenerar tokens o secretos que hayan sido expuestos en capturas o chats.

CRM.Data/Properties/launchSettings.json

Configuracion de ejecucion local en Visual Studio.

Define:

- Perfiles de arranque.
- URLs locales.
- Variables de entorno de desarrollo.

5. Controllers

Los controladores son los endpoints HTTP que consume el frontend o los webhooks externos.

AuthenticationController.cs

Maneja autenticacion.

Endpoints principales:

- Login.
- Logout.
- Usuario actual.

Seguridad:

- Usa cookies.
- Hashea contraseñas.
- Aplica bloqueo por intentos fallidos.
- Aplica rate limiting al login.

CrmManagementController.cs

Controlador central del CRM.

Responsabilidades:

- Contactos.
- Pipeline.
- Usuarios.
- Estados de conversacion.
- Tomar conversacion.
- Asignar conversacion.
- Actualizar perfil Meta.
- Actividad.
- Exportar contactos.
- Comentarios.
- Fallos.
- Reportes.

Seguridad:

- Requiere rol Administrador, Supervisor o Asesor.
- Usa CrmAccessService para filtrar clientes, conversaciones y mensajes cuando el usuario es asesor.
- Admin/supervisor ven todo.
- Asesor ve lo asignado y las conversaciones nuevas sin asignar que puede tomar; no ve conversaciones de otros asesores.

WhatsAppController.cs

Maneja WhatsApp y endpoints de conversaciones.

Responsabilidades:

- Verificación de webhook.
- Recepción de webhooks de WhatsApp.
- Consulta de conversaciones.
- Consulta de una conversación.
- Envío de mensajes.
- Indicador de escritura.
- Cambiar estado del bot por conversación.

Seguridad:

- Webhook de Meta permite acceso anónimo solo para verificación/recepción.
- Conversaciones y envío requieren autenticación.
- Asesor puede operar conversaciones asignadas y tomar conversaciones nuevas sin asignar; no puede operar conversaciones de otros asesores.

WhatsAppAttachmentsController.cs

Maneja envío de archivos desde conversaciones.

Responsabilidades:

- Validar tamaño máximo.
- Validar extensión permitida.
- Subir archivo a Cloudinary.
- Registrar mensaje de archivo.
- Enviar archivo por el canal cuando aplique.

Seguridad:

- Requiere autenticación.
- Valida acceso a la conversación.
- Limita tipos de archivo.
- Limita tamaño a 15 MB.

OportunidadesController.cs

Maneja oportunidades de venta.

Responsabilidades:

- Listar oportunidades.
- Exportar oportunidades.
- Resumen por etapa.
- Crear oportunidad.
- Cambiar etapa.
- Asignar oportunidad.

Seguridad:

- Admin/supervisor ven todo.
- Asesor solo ve oportunidades propias o vinculadas a clientes/conversaciones asignadas.
- Asesor no puede forzar asignación a otro usuario al crear.

TareasController.cs

Maneja tareas de seguimiento.

Responsabilidades:

- Listar tareas.
- Crear tarea.
- Completar tarea.
- Cancelar tarea.

Seguridad:

- Admin/supervisor ven todo.
- Asesor solo ve tareas propias o relacionadas con sus clientes/conversaciones.
- Asesor no puede crear tareas asignadas a otro usuario.

NotasController.cs

Maneja notas internas por cliente.

Responsabilidades:

- Listar notas.
- Crear notas.

Seguridad:

- El asesor solo puede ver o crear notas de clientes permitidos.
- Si la nota referencia una conversacion, tambien se valida acceso a esa conversacion.

EtiquetasController.cs

Maneja etiquetas.

Responsabilidades:

- Listar etiquetas.
- Crear etiquetas.
- Asignar etiqueta a cliente.
- Quitar etiqueta de cliente.
- Ver etiquetas de un cliente.

Seguridad:

- Crear etiquetas queda para admin/supervisor.
- Asignar o quitar etiquetas valida acceso al cliente.

BotController.cs

Maneja configuracion del bot.

Responsabilidades:

- Leer configuracion de bot.
- Guardar configuracion.

Seguridad:

- Restringido a Administrador y Supervisor.

PlantillasController.cs

Maneja plantillas rapidas para asesores.

Responsabilidades:

- Listar plantillas activas para todos los roles autenticados.
- Listar plantillas completas para administracion.
- Guardar plantillas rapidas.

Seguridad:

- Asesor puede leer plantillas activas.
- Administrador y supervisor pueden administrarlas.

IntegracionesController.cs

Maneja integraciones sociales.

Responsabilidades:

- Leer y guardar configuracion de canales.
- Copiar/mostrar webhooks.
- Ejecutar pruebas de inbound.
- Diagnosticar integraciones.
- Recibir webhook Meta para Facebook/Instagram.
- Recibir webhook TikTok preparado.

HealthController.cs

Endpoint de salud.

Sirve para:

- Verificar que la aplicacion responde.
- Usarse en monitoreo simple.

6. Data

CrmDbContext.cs

Es el DbContext de Entity Framework Core.

Define:

- Tablas.
- Llaves primarias.
- Columnas.
- Relaciones.
- Indices.
- Comportamiento de borrado.

DbSets:

- Clientes.
- Conversaciones.
- Mensajes.
- Usuarios.
- Etiquetas.

- ClienteEtiquetas.
- Oportunidades.
- Tareas.
- NotasInternas.
- ActividadLogs.

7. Models y base de datos

Cliente.cs / tabla crm_cliente

Representa un cliente.

Campos principales:

- nCliente: identificador.
- cNombre: nombre.
- cTelefono: telefono o identificador principal.
- cEmail: correo.
- cDocumento: documento.
- cFotoPerfilUrl: foto de perfil.
- cCanalOrigen: canal de origen.
- dFechaRegistro: fecha de creacion.
- cEstado: estado.

Relaciones:

- Tiene muchas conversaciones.

Conversacion.cs / tabla crm_conversacion

Representa un hilo de atencion.

Campos principales:

- nConversacion.
- nCliente.
- nUsuarioAsignado.
- cEstado.
- cCanal.
- cExternalThreadId.
- cBotEstado.
- dFechaInicio.
- dUltimoMensaje.
- dUltimoMensajeCliente.
- dBotPausadoDesde.
- nBotPausadoPor.

Relaciones:

- Pertenece a un cliente.
- Puede estar asignada a un usuario.
- Tiene muchos mensajes.

Mensaje.cs / tabla crm_mensaje

Representa un mensaje entrante o saliente.

Campos principales:

- nMensaje.
- nConversacion.
- cWhatsappld.
- cCanal.
- cExternalld.
- cDireccion: entrante o saliente.
- cTipo: texto, imagen, documento, bot, comentario.
- cMensaje.
- cEstado.
- dFecha.

CrmUsuario.cs / tabla crm_usuario

Representa usuario interno.

Campos principales:

- nUsuario.
- cUsuario.
- cNombre.
- cPasswordHash.
- cRol.
- cEstado.

Roles:

- Administrador.
- Supervisor.
- Asesor.

Etiqueta.cs y ClienteEtiqueta

Representan segmentacion de clientes.

Tablas:

- crm_etiqueta.
- crm_cliente_etiqueta.

Uso:

- Etiquetar clientes.
- Buscar o clasificar clientes.

Oportunidad.cs / tabla crm_oportunidad

Representa una venta potencial.

Campos:

- Cliente.
- Conversacion opcional.
- Usuario asignado.

- Titulo.
- Monto.
- Moneda.
- Etapa.
- Probabilidad.
- Fecha estimada.
- Fecha real de cierre.
- Motivo de perdida.

Tarea.cs / tabla crm_tarea

Representa un seguimiento.

Campos:

- Cliente opcional.
- Conversacion opcional.
- Oportunidad opcional.
- Titulo.
- Descripcion.
- Fecha de vencimiento.
- Estado.
- Asignado a.
- Creado por.

NotaInterna.cs / tabla crm_nota_interna

Representa una nota privada del equipo.

Campos:

- Cliente.
- Conversacion opcional.
- Texto.
- Usuario creador.
- Fecha.

ActividadLog.cs / tabla crm_actividad_log

Registra auditoria.

Campos:

- Entidad.
- EntidadId.
- Accion.
- Valor anterior.
- Valor nuevo.
- Usuario.
- Fecha.

CanalSocial.cs

Constantes y normalizacion de canales.

Canales:

- WhatsApp.
- Facebook.
- Instagram.
- TikTok.

8. Services

WhatsAppService.cs

Clase parcial base del servicio WhatsApp. Agrupa logica dividida en archivos parciales.

WhatsAppService.Inbound.cs

Procesa mensajes entrantes.

Flujo:

- Recibe mensaje del webhook.
- Evita duplicados.
- Busca o crea cliente.
- Busca o crea conversacion.
- Asigna automaticamente si aplica.
- Guarda mensaje.
- Actualiza fechas.
- Ejecuta bot si corresponde.

WhatsAppService.Outgoing.cs

Procesa mensajes salientes.

Flujo:

- Valida conversacion.
- Marca conversacion en atencion.
- Asigna usuario si responde.
- Pausa bot.
- Guarda mensaje.
- Envia a Meta en segundo plano si el envio real esta activo.

WhatsAppService.Queries.cs

Consulta conversaciones.

Incluye:

- Obtener una conversacion con cliente, asesor y mensajes.
- Obtener lista de conversaciones.
- Filtros opcionales por usuario asignado para asesores.

WhatsAppService.Assignment.cs

Maneja asignacion automatica y conversaciones pendientes.

WhatsAppService.BotState.cs

Maneja pausa/activacion del bot por conversacion.

WhatsAppService.Status.cs

Maneja estados de mensajes o callbacks de entrega.

WhatsAppService.Typing.cs

Maneja indicador de escritura.

WhatsAppCloudApiService.cs

Cliente HTTP para WhatsApp Cloud API.

Responsabilidades:

- Enviar textos.
- Enviar archivos.
- Descargar media si aplica.
- Validar si el envio real esta activo.

MetaGraphApiService.cs

Cliente HTTP para Graph API/Instagram Graph.

Responsabilidades:

- Consultar cuenta me.
- Consultar conversaciones de Instagram.
- Obtener perfil de contacto.
- Diagnosticar respuestas de Meta.

MetaMessagingService.cs

Servicio de envio para Facebook/Instagram cuando hay permisos y tokens configurados.

MetaWebhookService.cs

Procesa webhooks de Meta para Facebook/Instagram.

SocialInboundService.cs

Normaliza mensajes sociales entrantes.

Responsabilidades:

- Buscar o crear cliente social.
- Buscar o crear conversacion social.
- Guardar mensaje.
- Ejecutar bot social si aplica.

SocialIntegrationService.cs

Maneja configuraciones de canales sociales.

Lee y guarda:

- WhatsApp.

- Instagram.
- Facebook.
- TikTok.

Archivo usado:

- App_Data/social-integrations.json.

BotSettingsService.cs

Maneja configuracion del bot.

Archivo usado:

- App_Data/bot-whatsapp.json.

QuickReplyTemplatesService.cs

Maneja plantillas rapidas separadas del bot.

Archivo usado:

- App_Data/quick-replies.json.

Regla:

- El bot automatiza.
- Las plantillas rapidas ayudan al asesor a responder manualmente.

CloudinaryStorageService.cs

Sube archivos a Cloudinary.

Devuelve:

- URL segura.
- Public ID.
- Tipo de recurso.

AuditoriaService.cs

Registra actividad en crm_actividad_log.

CrmAccessService.cs

Servicio central de permisos por pertenencia.

Responsabilidades:

- Saber usuario actual.
- Saber si el usuario es asesor.
- Saber si tiene acceso global.
- Filtrar conversaciones.
- Filtrar clientes.
- Filtrar mensajes.
- Filtrar oportunidades.
- Filtrar tareas.
- Validar acceso a conversacion, cliente, oportunidad y tarea.

Regla:

- Administrador y supervisor tienen acceso global.
- Asesor accede a lo asignado y a conversaciones nuevas sin asignar disponibles para tomar.

9. DTOs

DTOs/Bot

Define objetos para configurar bot:

- Opciones.
- Respuestas.
- Mensaje inicial.
- Configuración general.

DTOs/Social

Define objetos para integraciones sociales:

- Pruebas inbound.
- Guardado de configuración.

10. Extensions

ServiceCollectionExtensions.cs

Registra servicios en DI.

Incluye:

- Base de datos.
- Servicios internos.
- HttpClient externos.
- Autenticación por cookies.
- Rate limiting.
- CORS.

ApplicationBuilderExtensions.cs

Extensión de pipeline de aplicación.

SecurityHeadersExtensions.cs

Agrega headers de seguridad:

- X-Content-Type-Options.
- X-Frame-Options.
- Referrer-Policy.
- Permissions-Policy.
- X-Permitted-Cross-Domain-Policies.
- Content Security Policy.

RequestSecurityExtensions.cs

Protege solicitudes.

Incluye:

- no-store para respuestas /api.
- Bloqueo de peticiones cross-site peligrosas.
- Excepciones para webhooks externos.

11. Startup

DatabaseInitializer.cs

Inicializa base de datos y datos necesarios.

Puede:

- Aplicar migraciones.
- Crear usuario administrador inicial.
- Crear datos base si no existen.

12. Migrations

Carpeta con migraciones EF Core.

Archivos:

- 20260914235831_InicialCrm: base inicial.
- 20260915093000_BotPorConversacion: estado de bot por conversacion.
- 20260915103000_CanalesSociales: soporte multicanal.
- 20260916153000_ClienteFotoPerfil: foto de perfil de cliente.
- CrmDbContextModelSnapshot: snapshot del modelo actual.

13. wwwroot

Contiene frontend estatico.

index.html

Pantalla principal del CRM.

login.html

Pantalla de inicio de sesion.

MaterializeReference.css

Estilos principales heredados/base.

Script.js

Script historico o base.

js/api-session.js

Maneja sesion del usuario, modulo inicial, navegacion por rol y cierre de sesion.

js/module-router.js

Maneja navegacion interna y evita entrada visual a modulos no permitidos.

js/app.js

Arranque general del frontend.

js/state.js

Estado compartido del frontend.

js/utils.js

Funciones auxiliares.

js/inbox.js

Interfaz de Comunicaciones: lista conversaciones, abre chats, renderiza mensajes, muestra plantillas rapidas y envia respuestas.

js/messaging-attachments.js

Maneja adjuntos: seleccion, validacion, envio y renderizado.

js/contacts.js

Interfaz de Contactos: lista, busca, crea y abre ficha de cliente.

js/customer-panel.js

Ficha 360 del cliente: datos, notas, tareas, oportunidades y etiquetas.

js/tasks-sales.js

Modulo de tareas y ventas: seguimientos y oportunidades.

js/pipeline.js

Vista de pipeline de conversaciones.

Para asesores, el Pipeline funciona como tablero filtrado: muestra sus conversaciones y las nuevas sin asignar disponibles para tomar. Para administradores y supervisores funciona como tablero operativo global con controles de asignacion.

Incluye movimiento de tarjetas con drag and drop entre etapas.

js/dashboard.js

Dashboard ejecutivo.

js/activity-reports.js

Reportes, actividad y fallos.

js/bot.js

Configuracion del bot.

js/connections.js

Configuracion de integraciones.

js/users.js

Administracion de usuarios.

js/layout-profile.js

Comportamiento visual del perfil/layout.

js/quality-modules.js

Modulos de calidad operativa complementaria.

14. App_Data

bot-whatsapp.json

Configuracion local del bot.

Contiene:

- Mensaje inicial.
- Opciones.
- Respuestas.
- Limites.

quick-replies.json

Configuracion local de plantillas rapidas para asesores.

Contiene:

- Titulo.
- Mensaje.
- Categoria.
- Estado activo/inactivo.

social-integrations.json

Configuracion local de integraciones.

Puede contener:

- App IDs.
- Versiones de API.
- Tokens.
- Verify tokens.
- IDs de cuenta/pagina.

Recomendacion:

- No subir este archivo con secretos reales.
- Usar variables de entorno o secret manager en produccion.

15. Flujo de autenticacion

1. Usuario entra a login.html.
2. Frontend llama al endpoint de login.

3. Backend busca usuario activo.
4. Backend valida password con hash.
5. Si es valido, genera cookie segura.
6. Frontend consulta /api/auth/me.
7. Se cargan modulos segun rol.

Seguridad:

- Cookie HttpOnly.
- Cookie Secure.
- SameSite=Lax.
- Expiracion controlada.
- Bloqueo por intentos fallidos.
- Rate limiting.

16. Flujo de WhatsApp entrante

1. Cliente escribe por WhatsApp.
2. Meta envia webhook al endpoint publico.
3. ngrok o tunel reenvia al backend local.
4. Backend valida y procesa payload.
5. Busca o crea cliente.
6. Busca o crea conversacion.
7. Guarda mensaje entrante.
8. Actualiza ultimo mensaje.
9. Asigna asesor si aplica.
10. Ejecuta bot si corresponde.
11. Frontend muestra la conversacion.

17. Flujo de mensaje saliente

1. Asesor escribe desde Comunicaciones.
2. Frontend llama al endpoint de envio.
3. Backend valida acceso a la conversacion.
4. Backend pausa bot.
5. Backend registra mensaje saliente.
6. Si envio real esta activo, manda a Meta.
7. Estado queda registrado.

18. Flujo de archivos

1. Asesor selecciona archivo.
2. Frontend envia multipart al backend.
3. Backend valida extension y tamano.
4. Backend valida acceso a conversacion.
5. Backend sube archivo a Cloudinary.
6. Backend guarda mensaje de archivo.

7. Backend envia por canal si aplica.

19. Flujo de Instagram/Facebook

Estado actual:

- Configuración preparada.
- Webhook Meta preparado.
- Perfil de contacto preparado.
- Instagram Login validado para cuenta profesional.

Limitación:

- En modo desarrollador Meta no entrega DMs reales externos como producción.
- Se requiere publicación/capability/App Review para DMs reales y webhooks reales.

20. Seguridad implementada

Autenticación y roles

- Login obligatorio.
- Roles: Administrador, Supervisor, Asesor.
- Password hash.
- Cookies endurecidas.
- Endpoints con [Authorize].

Rate limiting

- Login: límite por usuario/IP.
- APIs internas: límite por usuario/IP.
- Respuesta 429 JSON.

Headers

- X-Content-Type-Options.
- X-Frame-Options.
- Referrer-Policy.
- Permissions-Policy.
- Content Security Policy básica.

Protección cross-site

- Bloqueo de métodos peligrosos cross-site.
- Excepciones para webhooks externos.

Control por asesor

Implementado en backend con CrmAccessService.

Protege:

- Clientes.
- Conversaciones.
- Mensajes.
- Comentarios.

- Archivos.
- Plantillas rapidas.
- Notas.
- Etiquetas por cliente.
- Tareas.
- Oportunidades.

Cache

- Respuestas /api con no-store.

21. Riesgos y recomendaciones de produccion

Riesgos:

- Secretos en appsettings.json.
- Tokens expuestos en capturas.
- ngrok como tunel temporal.
- CRM conectado directamente al ERP con credenciales amplias.
- Falta de 2FA.
- Falta de backups probados.

Recomendaciones:

- Rotar secretos expuestos.
- Mover secretos a variables de entorno o secret manager.
- Usar Cloudflare Tunnel o hosting formal.
- Usar Cloudflare Access con 2FA.
- Usar WAF y rate limiting externo.
- Separar CRM y ERP.
- No conectar directo a la base del ERP.
- Crear API puente para ERP con permisos minimos.
- Backups automaticos y prueba de restauracion.
- Monitoreo de logs y errores.

22. Flujo seguro recomendado con ERP

No recomendado:

- CRM expuesto con acceso directo amplio a la base del ERP.

Recomendado:

1. Usuario.
2. Cloudflare Access/WAF/2FA.
3. CRM Web.
4. API del CRM.
5. API puente ERP.
6. Base de datos ERP en red privada.

La API puente debe permitir solo operaciones controladas:

- Consultar cliente.
- Consultar stock.

- Consultar precios.
- Crear cotizacion.
- Crear pedido.
- Consultar estado de pedido.

No debe permitir:

- Operaciones administrativas.
- Borrados masivos.
- DROP.
- ALTER.
- Lectura libre de tablas sensibles.

23. Validaciones tecnicas usadas

Comandos usados para validar:

```
Get-ChildItem CRM.Data\wwwroot\js\*.js | Sort-Object Name | ForEach-Object { node --check $_.FullName }
dotnet build CRM.Data\CRM.Data.csproj -o _codex_build_check\crmdata
```

Resultado reciente:

- JavaScript valido.
- Build correcto.
- 0 errores.
- 0 advertencias.

24. Pendientes tecnicos recomendados

- Mover secretos fuera de appsettings.json.
- Rotar App Secret y tokens expuestos.
- Agregar pruebas automatizadas de permisos por rol/pertenencia.
- Implementar Cloudflare Tunnel o hosting formal.
- Configurar Cloudflare Access/2FA.
- Crear API puente ERP.
- Mejorar exportaciones Excel/PDF.
- Agregar monitoreo y logs consultables.
- Definir politica de backups.
- Completar publicacion/revision Meta para Instagram Messaging.

25. Cierre

El CRM tiene una base funcional y mantenible:

- Backend ASP.NET Core.
- Frontend modular.
- Base de datos relacional.
- Seguridad inicial aplicada.
- Control de asesores desde backend.
- WhatsApp operativo.
- Canales sociales preparados segun limitaciones reales.
- Documentacion separada para uso y mantenimiento.